GitHub 使用说明（队友版）

面向：没怎么用过 Git / GitHub 的队员

目标：配好环境、接上 **AI**、以后改报告只需要说一句话

环境：Windows + 本项目仓库 `Surpass567/dachuang-monthly-report`

---

1 先理解三件事，后面全通

概念	打个比方	你要记住的
远程仓库	云端的共享文件夹	大家看到的是它
本地仓库	你电脑上的副本	你改的是它
commit（提交）	给当前状态拍张快照	改完要拍快照
push（推送）	把快照上传到云端	不上传别人看不到
pull（拉取）	把云端的快照下载下来	改之前先拉，不然会打架

一句话记住顺序：

```
先 pull（拿最新） -> 再改 -> 再 commit（拍快照） -> 再 push（上传）
```

最容易犯的错：不 pull 就直接改，改完推不上去，或者把别人的改动顶掉。

---

2 装 Git（10 分钟）

2.1 下载安装

打开 <https://git-scm.com/download/win>，下载 **64-bit Git for Windows Setup**，
一路点「下一步」即可（默认选项都对）。

2.2 验证安装

打开 **PowerShell**（开始菜单搜索 `PowerShell`），输入：

```
git --version
```

看到类似 `git version 2.53.0.windows.2` 就说明装好了。
如果提示「无法识别 **git**」：关掉 PowerShell 重新开一个（环境变量需要刷新）。

2.3 配置身份（必须，否则不能提交）

```
git config --global user.name " 你的名字拼音或英文名"
git config --global user.email " 你的 GitHub 注册邮箱"
```

这两行会被写进每一次提交记录。写成能认出来的，比如：

```
git config --global user.name "DongDonggood"
git config --global user.email " 你的邮箱 @example.com"
```

验证：

```
git config --global --list
```

2.4 三条推荐配置（Windows 必配）

```
# 1) 中文文件名正常显示（否则会显示成一串 \345\210\206 这种）
git config --global core.quotepath false

# 2) 换行符：检出时转 Windows 格式，提交时转回 LF
git config --global core.autocrlf true

# 3) 凭据管理器（Windows 上默认就有，确认一下）
git config --global credential.helper manager
```

为什么第 **2** 条重要：Windows 用 CRLF 换行，Linux/Mac 用 LF。
不配的话，每次提交都会看到一堆 `LF will be replaced by CRLF` 警告，
虽然不影响功能，但很烦。

2.5 首次登录 GitHub 账号

这一步不需要命令。第一次 `git push` 时，会弹出一个窗口：

```
Sign in with your browser  <- 点这个
```

浏览器会打开 GitHub 登录页，输入账号密码授权，Windows 会记住这个凭据。
以后就不用再登录了。
如果你没看到弹窗、或者推送一直要密码：
GitHub 现在不接受账号密码，必须用 **Personal Access Token**（**PAT**）。
生成方法见下面第 2.6 节。

2.6 生成访问令牌（PAT）

1. 打开 <https://github.com/settings/tokens>
2. 点 **Generate new token (classic)**
3. Note 随便写（如 `laptop`），Expiration 选 90 天或 No expiration
4. 勾选 **`repo`**（这是唯一必须的权限）
5. 点最下面 **Generate token**
6. 立刻复制那串 **`ghp_xxxx...`**——页面一关就再也看不到了

然后在 PowerShell 里推送时，用户名填你的 GitHub 用户名，密码粘贴这个 **token**。

3 把仓库拿到本地

3.1 先确认你有权限

这一步最容易卡住。
仓库属于 `Surpass567`（张昊洋）。你直接 clone 能下载，但推不上去，除非：

- 方式 **A**（推荐）：让张昊洋在

仓库 → `Settings` → `Collaborators` → `Add people` 里把你的 GitHub 账号加进来

- 方式 **B**：自己 Fork 一份，改完提 Pull Request（麻烦些）

加完协作者后，去你邮箱点接受邀请。

3.2 克隆

选一个你喜欢的目录，比如 `D:\work`，然后：

```
cd D:\work
git clone https://github.com/Surpass567/dachuang-monthly-report.git
cd dachuang-monthly-report
```

验证：

```
git status
```

看到 `On branch main` 和 `nothing to commit, working tree clean` 就对了。

3.3 目录长什么样

```
dachuang-monthly-report/
├── README.md
├── CONTRIBUTING.md          <- 团队约定，建议读一遍
├── check.py                 <- 自检脚本
├── template/
│   └── ydzjreport.cls       <- 文档类（编译用）
└── reports/
    ├── 逻辑月 1_ 基础建立 _v1_1.tex / .pdf
    ├── 逻辑月 2_ 指标与形态.tex / .pdf
    ├── ...
    ├── 中期报告.tex / .pdf
    └── figs/                <- 所有图片
        └── src/             <- 生成图片的脚本与数据
```

4 让 AI 帮你操作 GitHub（核心）

4.1 原理：AI 就是帮你敲命令

你不需要背 git 命令。**AI** 能直接在你的电脑上执行命令，
包括 `git clone` / `pull` / `commit` / `push`。
你要做的只有两件事：

1. 把仓库克隆到本地（第 2 节做过一次就行）
2. 告诉 **AI** 你想干什么

4.2 前置条件清单（缺一个都跑不通）

条件	怎么确认
Git 装好	`git --version` 有输出
身份配好	`git config --global user.name` 有输出
已登录 GitHub	手动推一次成功过（或已存 PAT）
是仓库协作者	能成功 push 一次
仓库在本地	有 `dachuang-monthly-report` 文件夹

关键：先手动成功推一次，让凭据存进 Windows。
之后 AI 替你推就不用再登录了。

4.3 直接说人话就行 —— 现成话术

把下面的话直接发给 AI 即可（把你的仓库路径换成实际的）：

你想做的事	直接这样说
拿最新代码	在 `D:\work\dachuang-monthly-report` 里拉取最新代码
看我改了什么	看看 `D:\work\dachuang-monthly-report` 里我改了哪些文件
提交并上传	把我改的逻辑月 `2_指标与形态.tex` 提交并推送到 `GitHub`
提交说明写好点	提交我的改动，提交信息写清楚改了什么
推送报错	推送报错了，帮我看看什么原因
编译并自检	在 `reports` 目录编译逻辑月 `2`，跑两遍，然后跑 `check.py`
图插不进去	报告里图片显示不出来，帮我查日志
找回删掉的文件	我不小心删了 `reports/xxx.tex`，从 `git` 历史里找回来
撤销没提交的修改	把 `xxx.tex` 恢复到上一次提交的样子
看历史版本	看看上周的逻辑月 `1` 长什么样

4.4 一次说清一件事（重要）

模糊的请求，**AI** 只能猜。对比：

不好	好
帮我弄一下	把 reports/逻辑月 3_形态与物理.tex 提交并推送
推上去	先拉取最新，再把我的改动提交（说明写" 补充网格无关性验证"）并推送
图坏了	逻辑月 2 第 3 张图编译后不见了，帮我查编译日志

一个标准模板：

```
在 D:\work\dachuang-monthly-report 里：
1. 先拉取最新代码
2. 我改了 reports/逻辑月 3_ 形态与物理.tex（补了网格无关性一节）
3. 编译两遍并跑 check.py，有问题先修
4. 全部通过后提交并推送，提交信息用中文写清改了什么
```

这样说，**AI** 能一口气做完，不用来回问。

4.5 AI 动手前会先问你（正常现象）

涉及这些操作时，AI 通常会先确认：

- 要不要覆盖哪些文件
- 提交信息怎么写
- 检测到冲突时怎么处理

不要嫌烦，这些确认是在保护你的工作。

4.6 你必须自己盯的一件事

AI 提交完，去 **GitHub** 网页上看一眼：

```
https://github.com/Surpass567/dachuang-monthly-report/commits/main
```

确认最新一条提交是你刚才做的。
这一步不能省——历史上出现过「以为推上去了其实没有」的情况。

---

5 自己敲命令（速查表）

不想麻烦 AI 的时候：

5.1 每日流程（照抄）

```
cd D:\work\dachuang-monthly-report

git pull                                # 1) 先拉最新
# ... 改文件 ...
```

```
git status                                  # 2) 看改了啥
git add -A                                  # 3) 全部加入暂存
git commit -m " 补充逻辑月 3 网格无关性验证"   # 4) 提交（中文说明）
git push                                    # 5) 推送
```

5.2 常用命令

命令	作用
`git status`	看当前改了哪些文件
`git diff`	看具体改了什么内容
`git log --oneline -10`	看最近 10 次提交
`git pull`	拉取最新
`git push`	推送
`git add 文件名`	只加入某个文件
`git add -A`	加入所有改动
`git commit -m " 说明"`	提交并写说明
`git checkout -- 文件名`	撤销某个文件的未提交修改
`git restore 文件名`	同上（新写法）
`git log -- 文件名`	看某个文件的历史

5.3 提交信息怎么写

用中文，说清做了什么。参考这个仓库已有的写法：

```
好：补充逻辑月 3 网格无关性验证
好：修正控制框图：公式渲染、字体与字号
好：整理 figs/：代码、数据与报告引用的图片分开存放

不好：update
不好：修改
不好：asdf
```

---

6 本仓库的具体规矩

6.1 编译报告

每份报告必须编译两遍（第一遍生成目录和引用，第二遍才稳定）：

```
cd reports
xelatex 逻辑月 2_ 指标与形态.tex
xelatex 逻辑月 2_ 指标与形态.tex
```

需要 `../template/ydzjreport.cls`。报找不到就把 `template\ydzjreport.cls` 复制到 `reports\` 下。

6.2 写完必须自检

```
cd reports
python ..\check.py
```

它会检查：括号配对、交叉引用、图片文件是否存在，并编译验证。
三项必须都是 **0**：

```
Overfull : 0    <- 超框（表格/图片出血）
Missing  : 0    <- 缺字（方框、乱码）
Error    : 0    <- 编译错误
```

不为 **0** 就别提交，先修。

6.3 图片放哪

```
reports/figs/          <- 所有图片（报告引用的）
reports/figs/src/      <- 生成脚本与中间数据（不直接引用）
```

命名：月 N_图 M_简述.png（例：月 2_图 1_变量结构.png）
引用：\includegraphics[width=0.8\textwidth]{figs/月 2_图 1_变量结构.png}
两个坑（都实测过）：

1. 图片找不到时 **LaTeX** 不会报错中断，只会留空——要查日志，别只看「**PDF** 生成了」
2. 路径不能用 **Windows** 反斜杠：D:\work\... 会报 Undefined control sequence，

必须写相对的 figs/图.png

6.4 别提交这些

.gitignore 已经挡掉大部分，但注意别手动 git add 这些：

```
*.aux  *.log  *.out  *.synctex.gz  *.toc   <- 编译中间产物
__pycache__/                              <- Python 缓存
_ 历史版本 _*/                               <- 临时展开的旧版本
```

6.5 提交内容

该提交	不该提交
.tex 源文件	编译中间产物
.pdf（改了就重新编译提交）	临时脚本 _xxx.py
figs/ 里的图片	个人笔记、草稿
README / 文档	很大的临时数据

7 常见报错怎么办

7.1 推送被拒绝

报错：

```
! [rejected]  main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do not have locally.
```

原因：别人先推了，你本地落后了。
处理（千万别加 **--force**）：

```
git pull
# 如果没冲突，直接再推
git push
```

7.2 拉取时冲突

报错：

```
CONFLICT (content): Merge conflict in reports/逻辑月 2_ 指标与形态.tex
```

原因：你和别人改了同一个文件的同一处。
处理：文件里会出现这种标记：

```
<<<<<<< HEAD
你写的内容
=======
别人写的内容
>>>>>>> origin/main
```

手工改成你想要的结果（把 `<<<<<<<`、`=======`、`>>>>>>>` 三行删掉，保留正确内容），然后：

```
git add 那个文件
git commit -m " 解决冲突：合并逻辑月 2 的改动"
git push
```

搞不定就把冲突文件发给 **AI**，让它帮你判断该保留哪部分。

7.3 一堆 `LF will be replaced by CRLF` 警告

原因：Windows 换行符问题。
处理：按第 2.4 节配 `core.autocrlf true`，之后会少很多。不影响功能，可以忽略。

7.4 中文文件名显示成 \345\210\206

处理：

```
git config --global core.quotepath false
```

7.5 提示「Please tell me who you are」

原因：没配身份。
处理：回到第 2.3 节，配 `user.name` 和 `user.email`。

7.6 提示文件太大（超过 100 MB）

GitHub 单文件上限 100 MB。不要提交大文件（如原始 CFD 结果、大压缩包）。
要传大文件：用网盘共享，或找张昊洋商量用 Git LFS。

7.7 我不小心删了文件，能找回来吗

能。 Git 记录了每一次提交的完整状态。

```
# 看这个文件的历史
git log --oneline -- 被删的文件

# 从某次提交恢复
git checkout 提交号 -- 被删的文件
```

或者直接跟 **AI** 说：我不小心删了 `reports/xxx.tex`，从 `git` 历史里找回来。

7.8 我改乱了，想全部回到上次提交的样子

```
git status                  # 先看清楚要丢掉什么
git checkout -- .           # [!] 会丢掉所有未提交的修改
```

[!] 这会永久丢掉你没提交的改动，执行前先确认。

---

8 危险操作黑名单

以下命令不要自己敲，也不要让 **AI** 随便敲：

命令	后果
`git push --force`	抹掉别人的提交，最严重
`git reset --hard`	丢掉所有未提交的修改
`git checkout -- .`	同上
`git clean -fd`	删掉所有未跟踪的文件（新文件全没）
删除 `.git` 文件夹	整个历史没了，只能重新 clone
`git rebase` 到不熟悉的分支	容易丢提交

如果 **AI** 要执行这些，它会先问你——一定要看清楚再回答。

---

9 每日流程卡（建议截图存手机）

```
┌──────────────────────────────────────┐
│ 开工                                 │
│   git pull                           │
│                                      │
│ 干活（改 tex / 换图 / 补内容）       │
│                                      │
│ 编译两遍                             │
│   xelatex 你的报告.tex   x2          │
│                                      │
│ 自检                                 │
│   python ..\check.py                 │
│   三项必须都是 0                     │
│                                      │
│ 提交                                 │
│   git add -A                         │
│   git commit -m " 中文说明改了什么"  │
│   git push                           │
│                                      │
│ 验收                                 │
│   去 GitHub 网页看最新提交是不是你   │
└──────────────────────────────────────┘
```

---

10 遇到问题怎么问

10.1 自己先做

1. 把完整报错复制下来（别截图，要文字）
2. 跑一下 `git status` 看当前状态

10.2 问 AI 的标准格式

```
我的环境：Windows，仓库在 D:\work\dachuang-monthly-report
我想做：把逻辑月 3 的修改推上去
报错原文：
（把完整报错粘进来）
已经试过：git pull（没有解决问题）
```

有这三样，几乎立刻能定位。

10.3 绝对不要做的事

- 不要自己 `--force`

- 不要删除 `.git`
- 不要把报错只描述成「推不上去」（要贴原文）
- 不要在没 pull 的情况下大量改文件

---

11 一页速记

我想做的事	命令 / 说法
拿最新代码	`git pull`
看我改了啥	`git status`
保存并上传	`git add -A` → `git commit -m "说明"` → `git push`
撤销某文件修改	`git checkout -- 文件名`
找回删掉的文件	`git checkout 提交号 -- 文件`
看历史	`git log --oneline -10`
编译	`xelatex 报告.tex`（跑两遍）
自检	`python ..\check.py`
实在搞不定	把报错原文发给 AI

最后一句：

> 先 **pull**，改完立刻 **push**。
>
> 拖得越久，冲突越多。

说明文档 *v1.0* ｜ *2026-10-22*
适用仓库：*Surpass567/dachuang-monthly-report*